

Algorithms and Satisfiability

2. Satisfiability, Part II: Clause Learning, CDCL, and Phase Transition

How to *Efficiently* Think About What is True or False

Álvaro Torralba



AALBORG UNIVERSITET

Spring 2025

Agenda

- ① Introduction
- ② DPLL Limitations: $\text{DPLL} = (\text{A Restricted Form of}) \text{ Resolution}$
- ③ *Why* Did Unit Propagation Yield a Conflict?
- ④ Clause Learning
- ⑤ Conflict-Driven Clause Learning
- ⑥ Phase Transitions: Where the *Really* Hard Problems Are
- ⑦ Conclusion

Reminder: Our Agenda for This Topic

- **Chapter 1:** Basic definitions and concepts; resolution; DPLL.
 - Sets up the framework. Resolution is the quintessential reasoning procedure underlying most successful solvers.
- **This Chapter:** Clause learning; CDCL; practical problem structure.
 - State-of-the-art algorithms for reasoning about propositional logic, and an important observation about how they behave.

SAT

The SAT Problem: Given a propositional formula φ , decide whether or not φ is satisfiable.

- The first problem proved to be **NP**-complete!
- φ is commonly assumed to be in CNF. This is without loss of generality, because any φ can in polynomial time be transformed into a satisfiability-equivalent CNF formula (cf. **Chapter 1**).
- Active research area, annual SAT conference, lots of tools etc.
available: <http://www.satlive.org/>
- Tools addressing SAT are commonly referred to as **SAT solvers**.

Reminder: To decide whether $\text{KB} \models \varphi$, decide satisfiability of $\theta := \text{KB} \cup \{\neg\varphi\}$: θ is unsatisfiable iff $\text{KB} \models \varphi$.

→ Deduction can be performed using SAT solvers.

Reminder: General Problem Solving using Logic

(some new problem)



model problem in logic $\mapsto$ use off-the-shelf reasoning tool



(its solution)

- “Any problem that can be formulated as reasoning about logic.”
- Very successful using propositional logic and modern solvers for SAT! (Propositional satisfiability testing, $\rightarrow$ **This Chapter.**)

Reminder: Conventions

Terminology and Notation

- A literal l is an atom or the negation thereof (e.g., $P, \neg Q$); the negation of a literal is denoted $\bar{l}$ (e.g., $\overline{\neg Q} = Q$).
- A clause C is a disjunction of literals. We identify C with the set of its literals (e.g., $P \vee \neg Q$ becomes $\{P, \neg Q\}$).
- We identify a CNF formula ψ with the set Δ of its clauses (e.g., $(P \vee \neg Q) \wedge R$ becomes $\{\{P, \neg Q\}, \{R\}\}$).
- The empty clause is denoted $\square$.

→ For the remainder of this chapter, we assume that the input is a set Δ of clauses.

Our Agenda for This Chapter

- **DPLL = (A Restricted Form of) Resolution:** How does this relate to what we did in the last chapter?
→ Mathematical understanding of DPLL.
- **Why Did Unit Propagation Yield a Conflict?** How can we analyze which mistakes were made in “dead” search branches?
→ Knowledge is power, see next.
- **Clause Learning:** How can we learn from our mistakes?
→ One of the key concepts, perhaps *the* key concept, underlying the success of SAT.
- **Phase Transitions: Where the Really Hard Problems Are:** Are *all* formulas “hard” to solve?
→ The answer is “no”. And in some cases we can figure out exactly when they are/aren’t hard to solve.

But – What About Local Search for SAT?

There's a wealth of research on local search for SAT, e.g.:

GSAT Algorithm

INPUT: a set of clauses Δ , MAX-FLIPS, and MAX-TRIES

OUTPUT: a satisfying truth assignment of Δ , if found

for $i := 1$ to MAX-TRIES

$I :=$ a randomly-generated truth assignment

for $j := 1$ to MAX-FLIPS

if I satisfies Δ **then return** I

$X :=$ a proposition reversing whose truth assignment gives
 the largest increase in the number of satisfied clauses

$I := I$ with the truth assignment of X reversed

end for

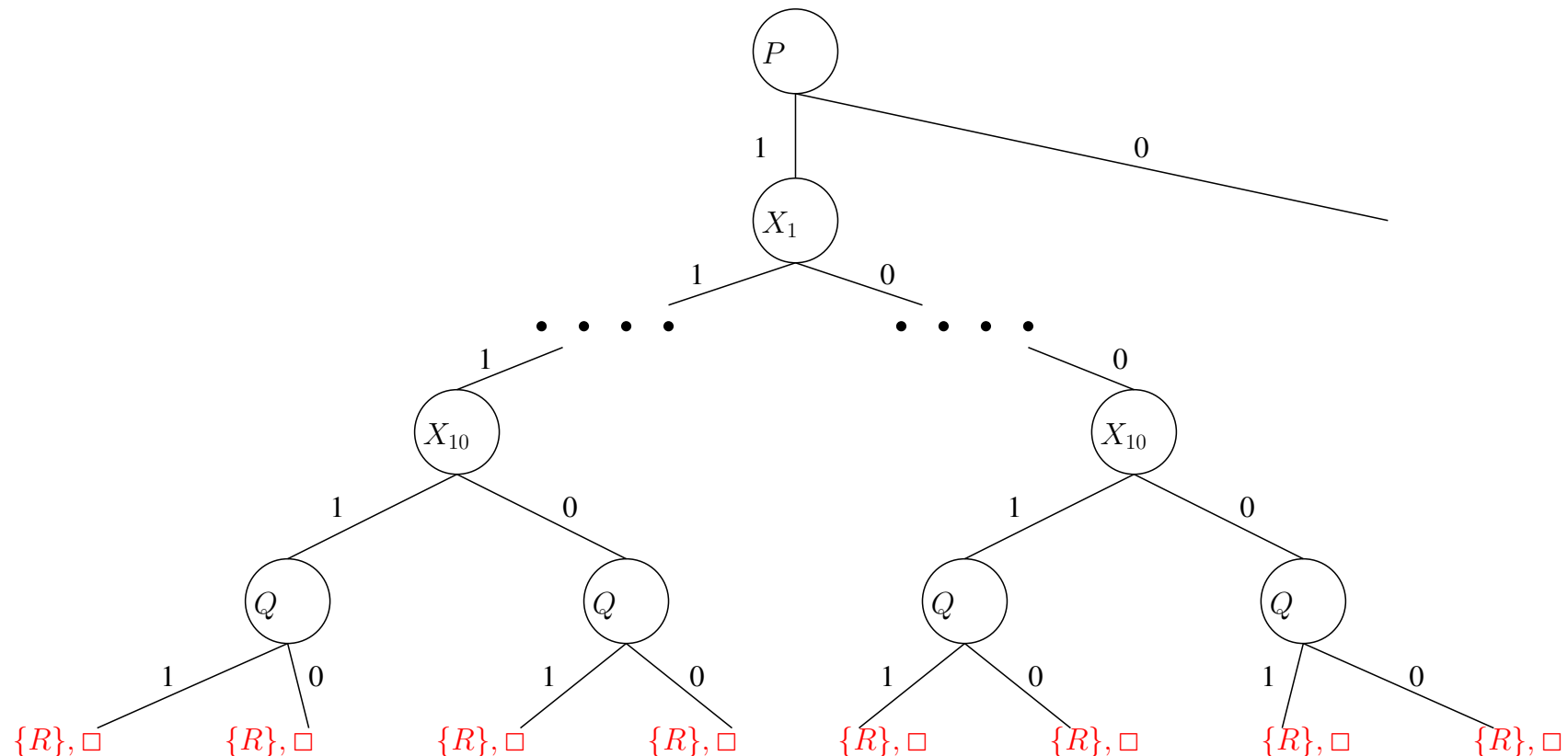
end for

return “no satisfying assignment found”

→ Local search is not as successful in SAT applications, though it can be a good complement to the techniques presented here. Not covered here.

Same Mistakes over Again: Example (Redundance1)

$$\Delta = \{\{\neg P, \neg Q, R\}, \{\neg P, \neg Q, \neg R\}, \{\neg P, Q, R\}, \{\neg P, Q, \neg R\}, \\ \{X_1, \dots, X_{100}\}, \{\neg X_1, \dots, \neg X_{100}\}\}$$



Note: Here, the problem could be avoided by splitting over different variables. **This is not so in general!**

DPLL = (A Restricted Form of) Resolution

Notation: Define the **number of decisions** of a DPLL run as the total number of times a truth value was set by either unit propagation or the splitting rule.

Theorem. *If DPLL returns “unsatisfiable” on Δ , then $\Delta \vdash \square$ with a resolution derivation whose length is at most the number of decisions.*

Proof Sketch. Consider first DPLL without the unit propagation rule.

Consider any leaf node N , for proposition X , both of whose truth values directly result in a clause C that has become empty.

Then for $X = 0$ the respective clause C must contain X ; and for $X = 1$ the respective clause C must contain $\neg X$. Thus we can resolve these two clauses to a clause $C(N)$ that does not contain X .

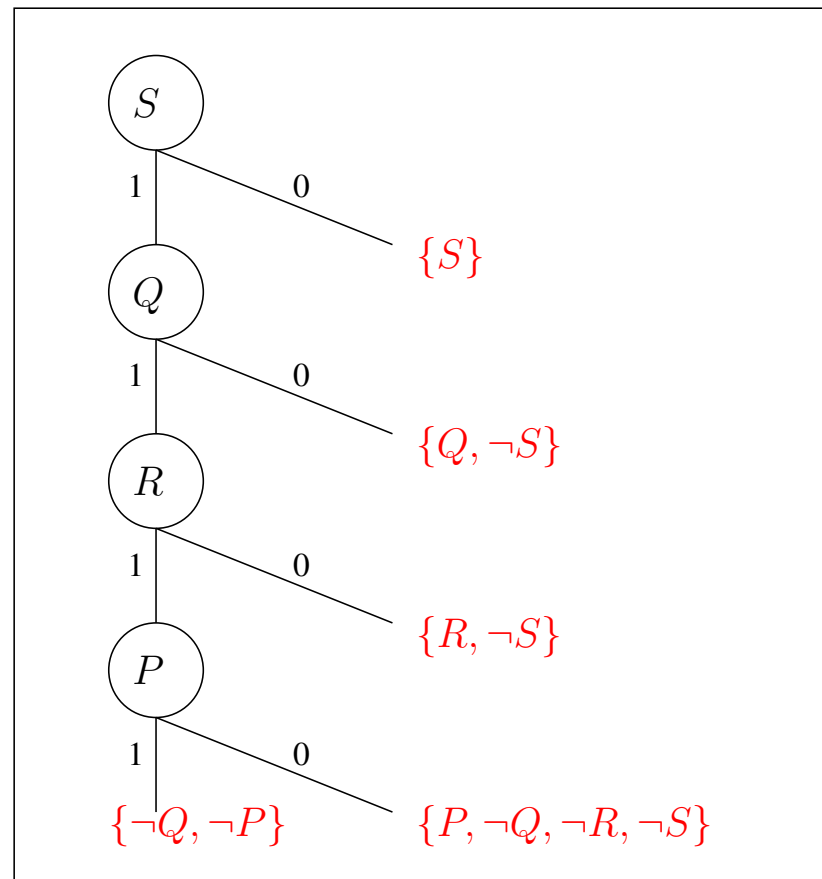
$C(N)$ can contain only the negations of the decision literals $l_1, \dots, l_k$ above N . Remove N from the tree, then iterate the argument. Once the tree is empty, we have derived the empty clause.

Unit propagation can be simulated via applications of the splitting rule, choosing a proposition that is constrained by a unit clause: One of the two truth values then immediately yields an empty clause.

DPLL = (A Restricted Form of) Resolution

Example: $\Delta = \{\{\neg Q, \neg P\}, \{P, \neg Q, \neg R, \neg S\}, \{Q, \neg S\}, \{R, \neg S\}, \{S\}\}$

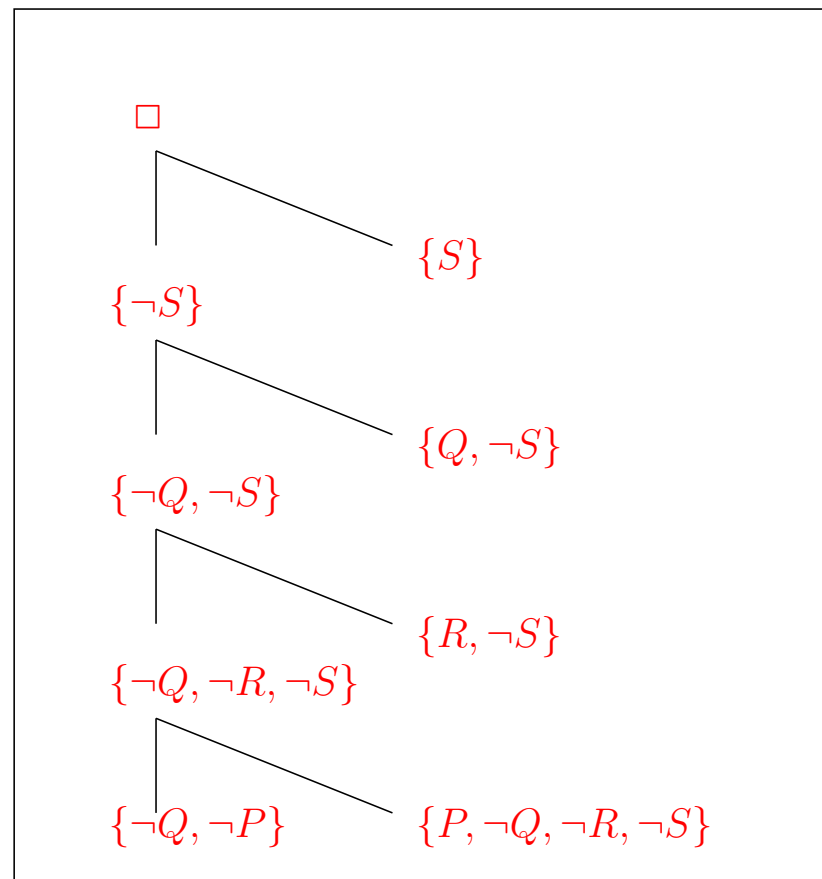
DPLL: (Without UP; leaves annotated with clauses that became empty)



DPLL = (A Restricted Form of) Resolution

Example: $\Delta = \{\{\neg Q, \neg P\}, \{P, \neg Q, \neg R, \neg S\}, \{Q, \neg S\}, \{R, \neg S\}, \{S\}\}$

Resolution Proof from that DPLL Tree:



DPLL vs. Resolution: Discussion

So What? The theorem we just proved helps to *understand* DPLL:

→ DPLL is an effective practical method for conducting resolution proofs.

In Fact:

- DPLL = **tree resolution**.
- This is a fundamental weakness! There are inputs Δ whose shortest tree-resolution proof is exponentially longer than their shortest (general) resolution proof.
 - In a tree resolution, each derived clause C is used only once (at its parent). **The same C is derived anew every time it is used!**

→ DPLL “makes the same mistakes over and over again”.

→ To the rescue: **clause learning**.

How To *Not* Make the Same Mistakes Over Again?

... it's not that difficult, really:

- Ⓐ Figure out what went wrong.
- Ⓑ Learn to not do that again in the future.

And now for DPLL:

Ⓐ *Why Did Unit Propagation Yield a Conflict?*

→ This section. We will capture the “what went wrong” in terms of graphs over literals set during the search, and their dependencies.

Ⓑ *What can we learn from that information?*

→ A new clause! Next section.

Implication Graphs

Notation/Terminology: Literals set along a branch of DPLL

- Value of P set by the splitting rule: choice literal, P for $I(P) = 1$, respectively $\neg P$ for $I(P) = 0$.
- Value of P set by the UP rule: implied literal P respectively $\neg P$.
- Empty clause derived by UP: conflict literal $\square$.

Definition (Implication Graph). Let Δ be a set of clauses, and consider any search branch β of DPLL on Δ . The *implication graph* G^{impl} is a directed graph. Its vertices are the choice and implied literals along β , as well as a separate conflict vertex $\square_C$ for every clause C that became empty.

Where $\{l_1, \dots, l_k, l'\} \in \Delta$ became unit with implied literal l' , G^{impl} includes the arcs $\overline{l_1} \rightarrow l', \dots, \overline{l_k} \rightarrow l'$. Where $C = \{l_1, \dots, l_k\} \in \Delta$ became empty, G^{impl} includes the arcs $\overline{l_1} \rightarrow \square_C, \dots, \overline{l_k} \rightarrow \square_C$.

- How do we know that $\overline{l_1}, \dots, \overline{l_k}$ are vertices in G^{impl} : Because $\{l_1, \dots, l_k, l'\}$ became unit respectively empty.
- Vertices with indegree 0: Choice literals, and unit clauses of Δ .

Implication Graphs: Example (Vanilla1) in Detail

$$\Delta = \{\{P, Q, \neg R\}, \{\neg P, \neg Q\}, \{R\}, \{P, \neg Q\}\}$$

UP ($R \mapsto 1$)

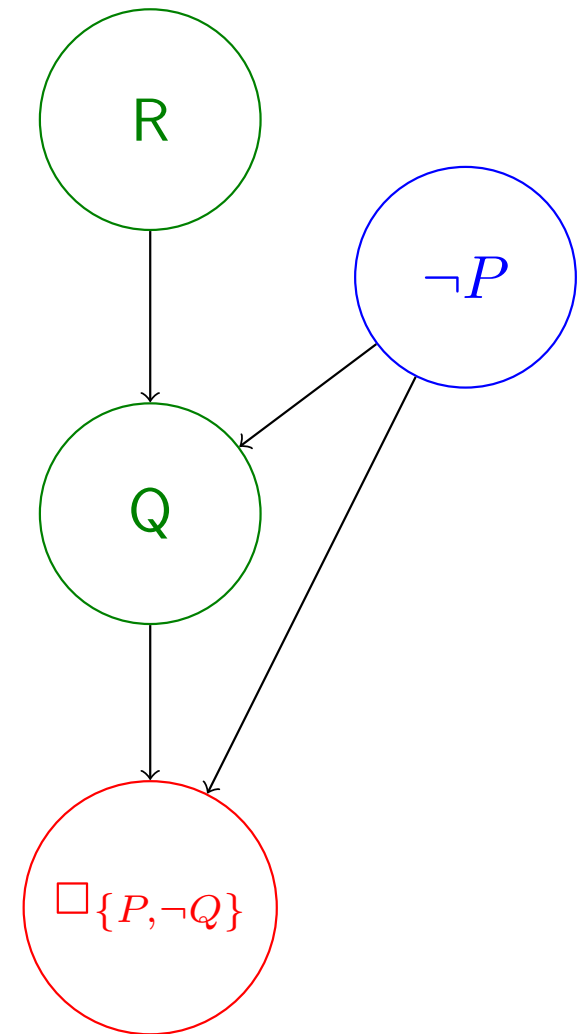
$$\{\{P, Q\}, \{\neg P, \neg Q\}, \{P, \neg Q\}\}$$

SP ($P \mapsto 0$)

$$\{\{Q\}, \{\neg Q\}\}$$

UP ($Q \mapsto 1$)

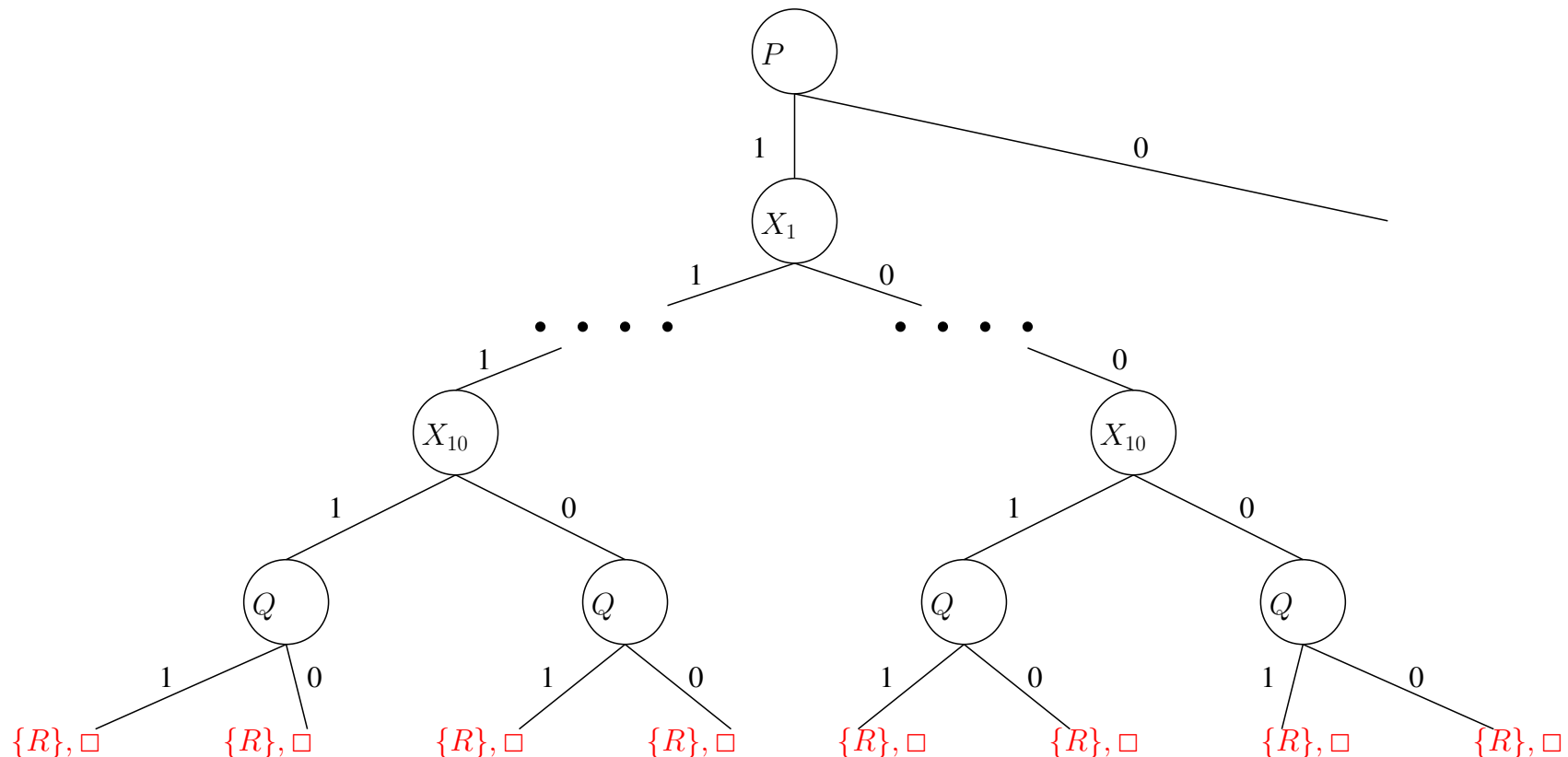
$$\{\square\}$$



Implication Graphs: Example (Redundance1)

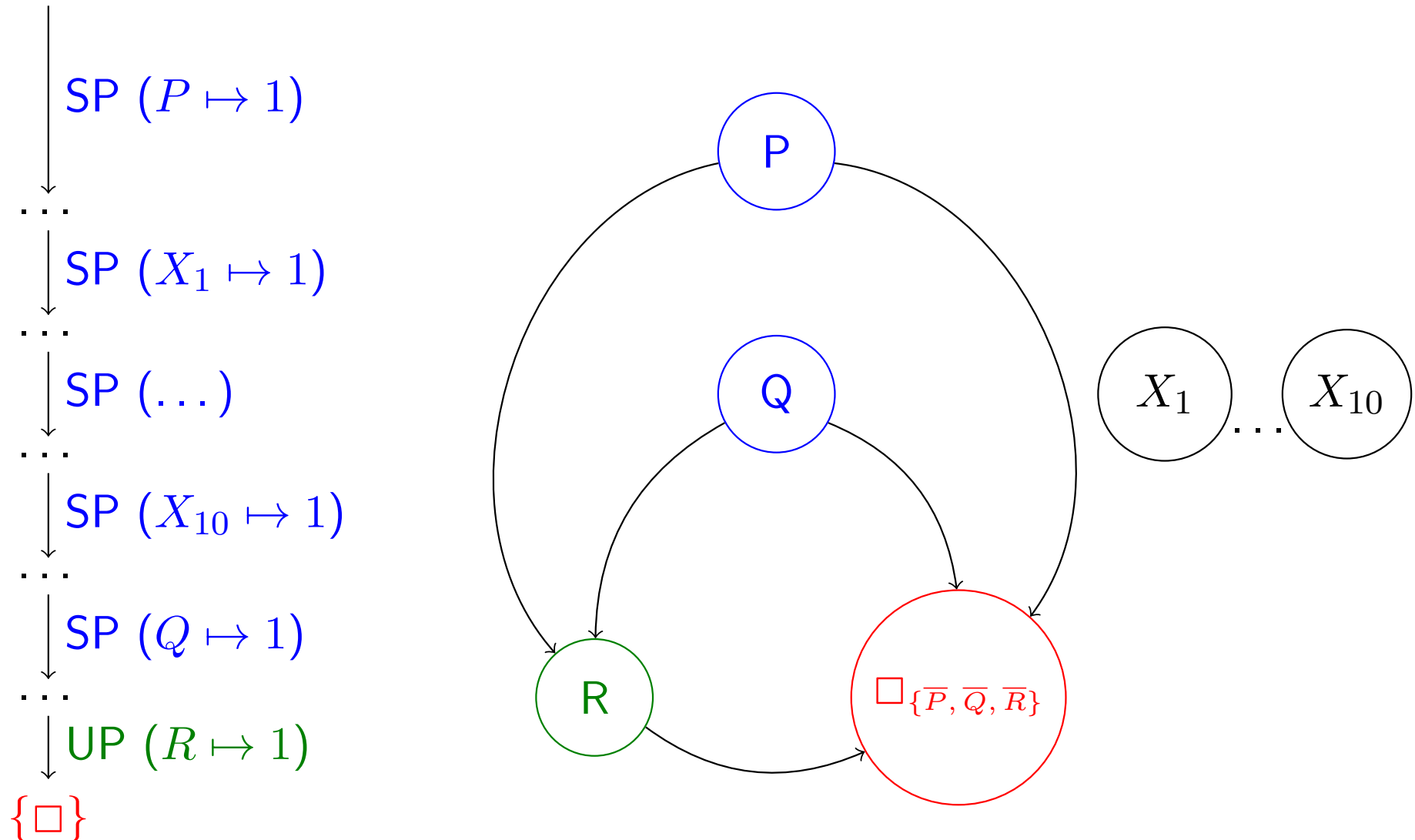
$$\Delta = \{\{\neg P, \neg Q, R\}, \{\neg P, \neg Q, \neg R\}, \{\neg P, Q, R\}, \{\neg P, Q, \neg R\}, \\ \{X_1, \dots, X_{100}\}, \{\neg X_1, \dots, \neg X_{100}\}\}$$

Choice: $P, X_1, \dots, X_{10}, Q$. Implied: R .



Implication Graphs: Example (Redundance1)

$$\Delta = \{\{\neg P, \neg Q, R\}, \{\neg P, \neg Q, \neg R\}, \{\neg P, Q, R\}, \{\neg P, Q, \neg R\}, \{X_1, \dots, X_{100}\}, \{\neg X_1, \dots, \neg X_{100}\}\}$$



Implication Graphs: Example (Redundance2)

$$\Delta = \{ \{ \neg P, \neg Q, R \}, \{ \neg P, \neg Q, \neg R \}, \{ \neg P, Q, R \}, \{ \neg P, Q, \neg R \}, \\ \{ \neg Q, S \}, \{ \neg Q, \neg S \}, \{ X_1, \dots, X_{100} \}, \{ \neg X_1, \dots, \neg X_{100} \} \}$$

↓ SP ($P \mapsto 1$)

⋮

↓ SP ($X_1 \mapsto 1$)

⋮

↓ SP (...)

⋮

↓ SP ($X_{10} \mapsto 1$)

⋮

↓ SP ($Q \mapsto 1$)

⋮

↓ UP ($R \mapsto 1$)

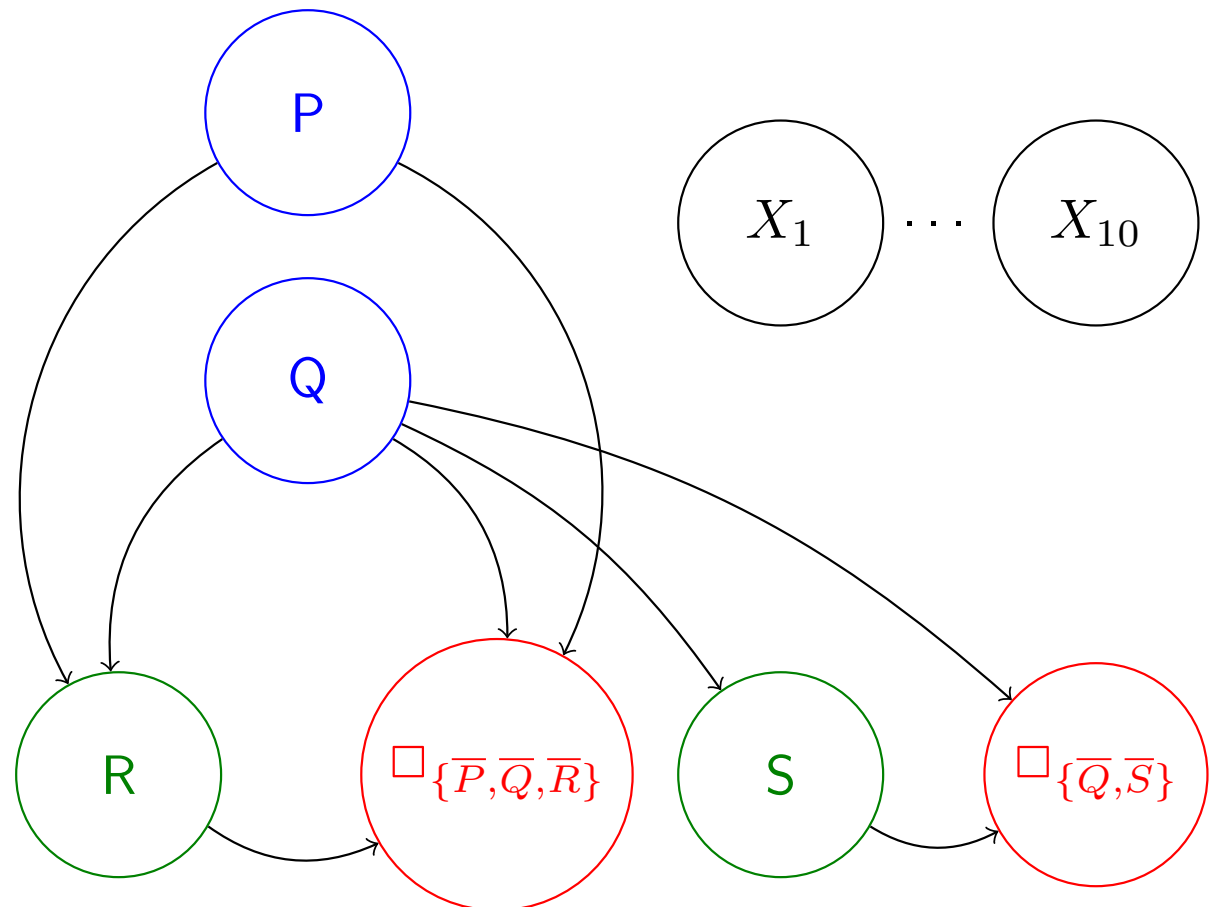
⋮

↓ UP ($S \mapsto 1$)

⋮

{□}

Choice: $P, X_1, \dots, X_{10}, Q$. Implied: R, S .



Questionnaire

Recall: Where $\{l_1, \dots, l_k, l'\} \in \Delta$ became unit on search branch β , with implied literal l' , G^{impl} includes the arcs $\bar{l}_1 \rightarrow l', \dots, \bar{l}_k \rightarrow l'$.

Question!

Can implication graphs have cycles?

(A): Yes

(B): No

→ No, because the implication graph keeps track of chronological behavior along the current DPLL search branch β . Unit propagation cannot derive l' whose value was already set beforehand.

In detail: Assume there is a cycle. Consider the first time point along the search branch where the cycle occurs, because an arc $(\bar{l}_i, l')$ is added to the implication graph. Then (a) l' is implied because $\{l_1, \dots, l_k, l'\} \in \Delta$ just became unit; and (b) as $(\bar{l}_i, l')$ closes a cycle, l' must already have been in the implication graph beforehand. (a) and (b) are in contradiction.

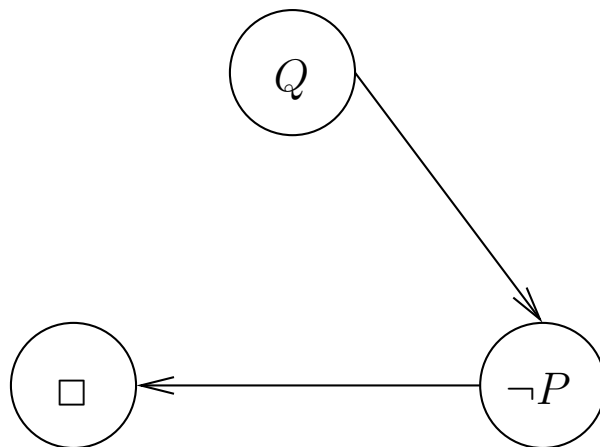
Implication Graphs: A Remark

→ The implication graph is *not* uniquely determined by the choice literals.

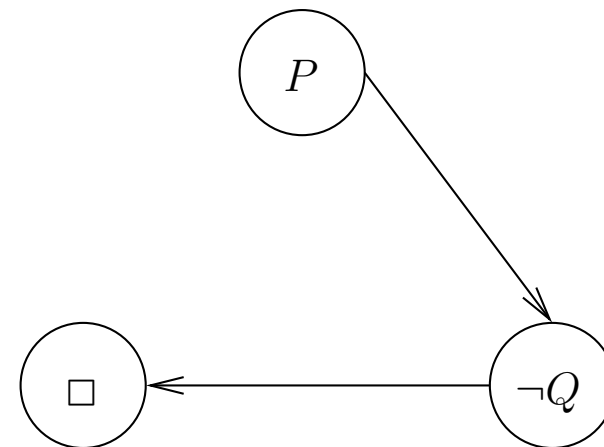
Because:

- The implication graph also depends on “ordering decisions” made during UP: Which unit clause is picked first.
- Example: $\Delta = \{\{\neg P, \neg Q\}, \{Q\}, \{P\}\}$

Option 1:



Option 2:



Conflict Graphs

→ A conflict graph captures “what went wrong” in a failed node.

Definition (Conflict Graph). Let Δ be a set of clauses, and let G^{impl} be the implication graph for some search branch of DPLL on Δ . A *conflict graph* G^{confl} is a **sub-graph** of G^{impl} induced by a subset of vertices such that:

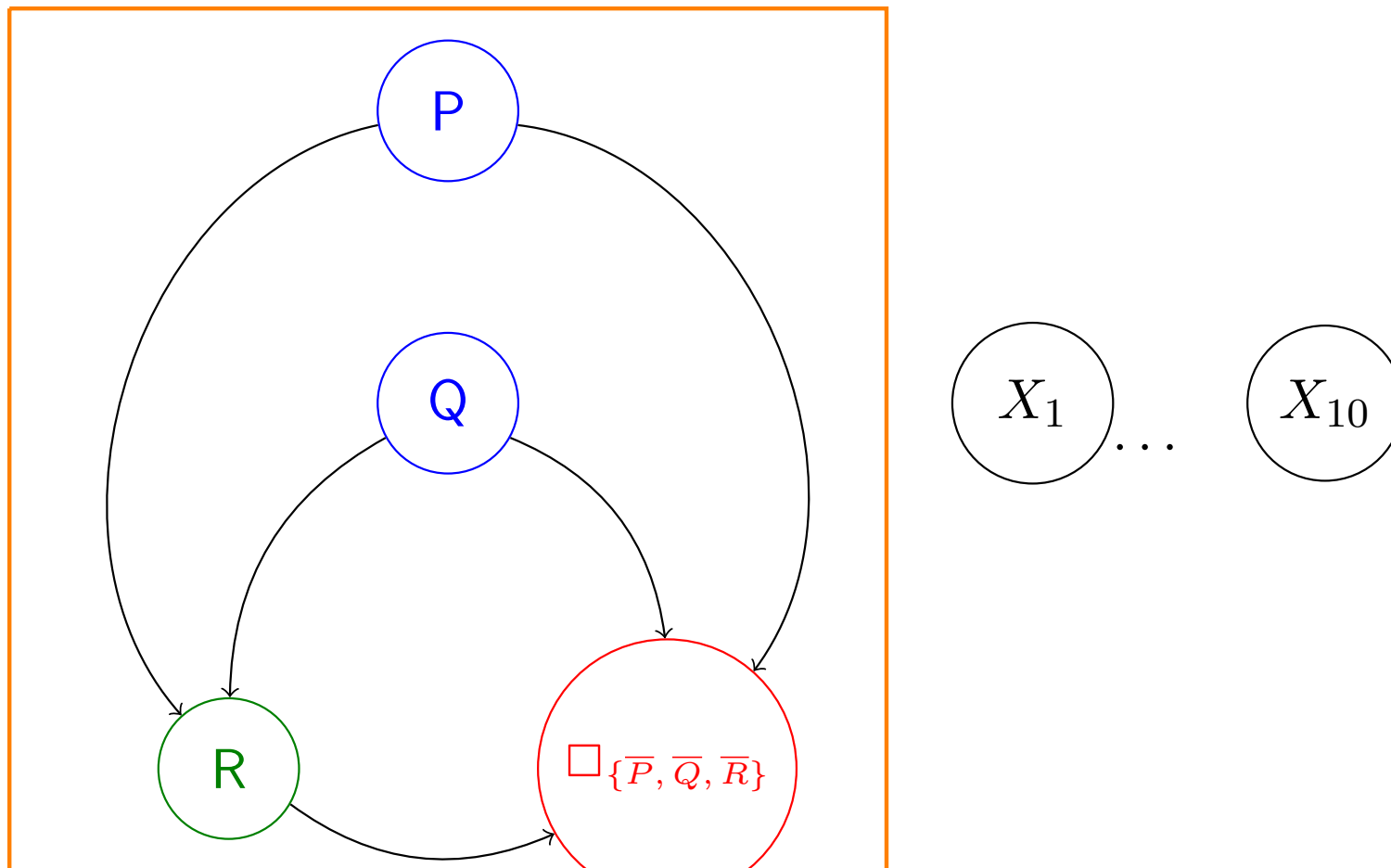
- (i) G^{confl} contains exactly one conflict vertex $\square_C$.
- (ii) If l' is a vertex in G^{confl} , then all parents of l' , i.e. vertices $\bar{l}_i$ with a G^{impl} arc $(\bar{l}_i, l')$, are vertices in G^{confl} as well.
- (iii) All vertices in G^{confl} have a path to $\square_C$.

→ Conflict graph = Starting at a conflict vertex, backchain through the implication graph until reaching choice literals.

Conflict Graphs: Example (Redundance1)

$$\Delta = \{\{\neg P, \neg Q, R\}, \{\neg P, \neg Q, \neg R\}, \{\neg P, Q, R\}, \{\neg P, Q, \neg R\}, \\ \{X_1, \dots, X_{100}\}, \{\neg X_1, \dots, \neg X_{100}\}\}$$

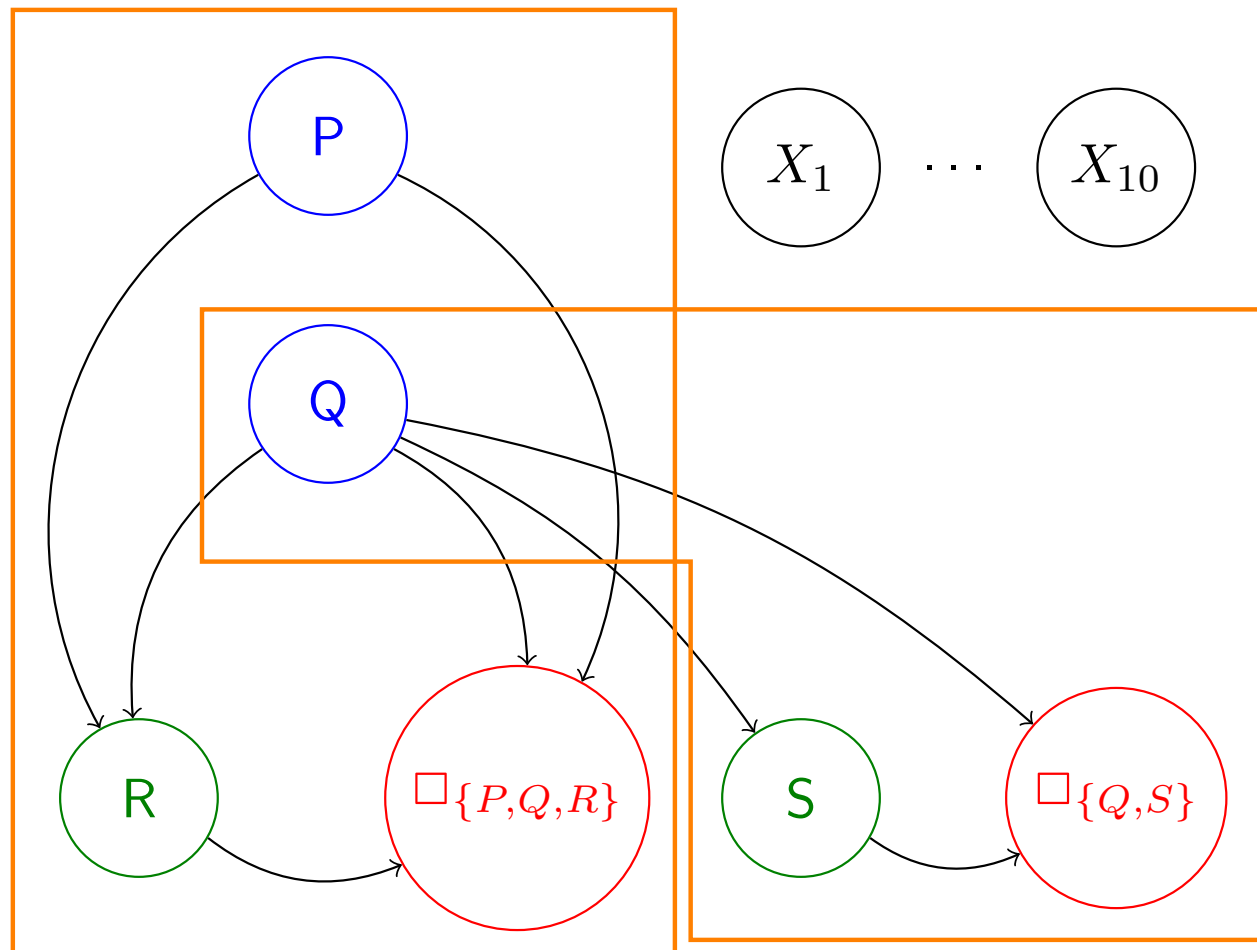
Choice: $P, X_1, \dots, X_{10}, Q$. Implied: R .



Conflict Graphs: Example (Redundance2)

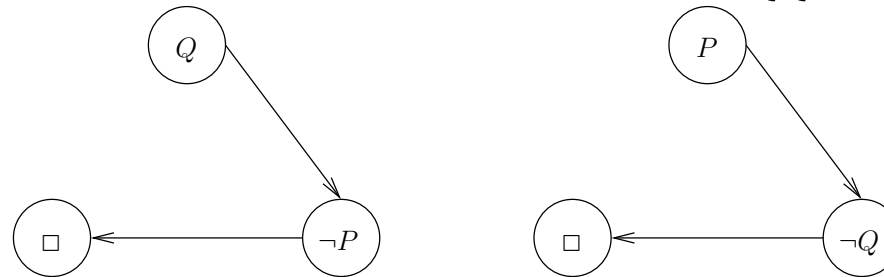
$\Delta = \{\{\neg P, \neg Q, R\}, \{\neg P, \neg Q, \neg R\}, \{\neg P, Q, R\}, \{\neg P, Q, \neg R\},$
 $\{\neg Q, S\}, \{\neg Q, \neg S\}, \{X_1, \dots, X_{100}\}, \{\neg X_1, \dots, \neg X_{100}\}\}$

Choice: $P, X_1, \dots, X_{10}, Q$. Implied: R, S .



Questionnaire

Recall: The implication graph depends on “ordering decisions” during UP: Which unit clause is picked first. E.g. $\Delta = \{\{\neg P, \neg Q\}, \{Q\}, \{P\}\}$.



Question!

Does the **existence** of a conflict graph depend on these decisions?

(A): Yes

(B): No

→ Observe: A conflict graph exists iff $\square$ is UP-derivable in the current simplified formula Δ' . So the question is whether it can happen that, when propagating a unit clause $\{l\}$ in Δ' , on the resulting simplified Δ'' the UP calculus cannot derive the empty clause anymore.

The answer is no. Δ'' can be obtained in two steps: 1. Remove $\bar{l}$ from every $C \in \Delta'$ where $\bar{l} \in C$ to obtain Δ'_l . 2. Remove $C \in \Delta'_l$ where $l \in C$ to obtain Δ'' . 1. cannot hurt $\square$ -derivability because every clause of Δ'_l is a sub-clause of Δ' , and smaller clauses can only be better. 2. cannot hurt $\square$ -derivability because $\bar{l}$ is not contained in Δ'_l (so if $l \in C$ then no derivative of C can ever become empty).

Clause Learning

Observe: Conflict graphs encode *logical entailments*

$$\Delta \models \left(\bigwedge_{l \in \text{choiceLits}(G^{\text{confl}})} l \right) \rightarrow \perp$$

→ Given Δ , setting all choice literals in a conflict graph results in failure.

Observe: We can re-write this!

$$\Delta \models \bigvee_{l \in \text{choiceLits}(G^{\text{confl}})} \bar{l}$$

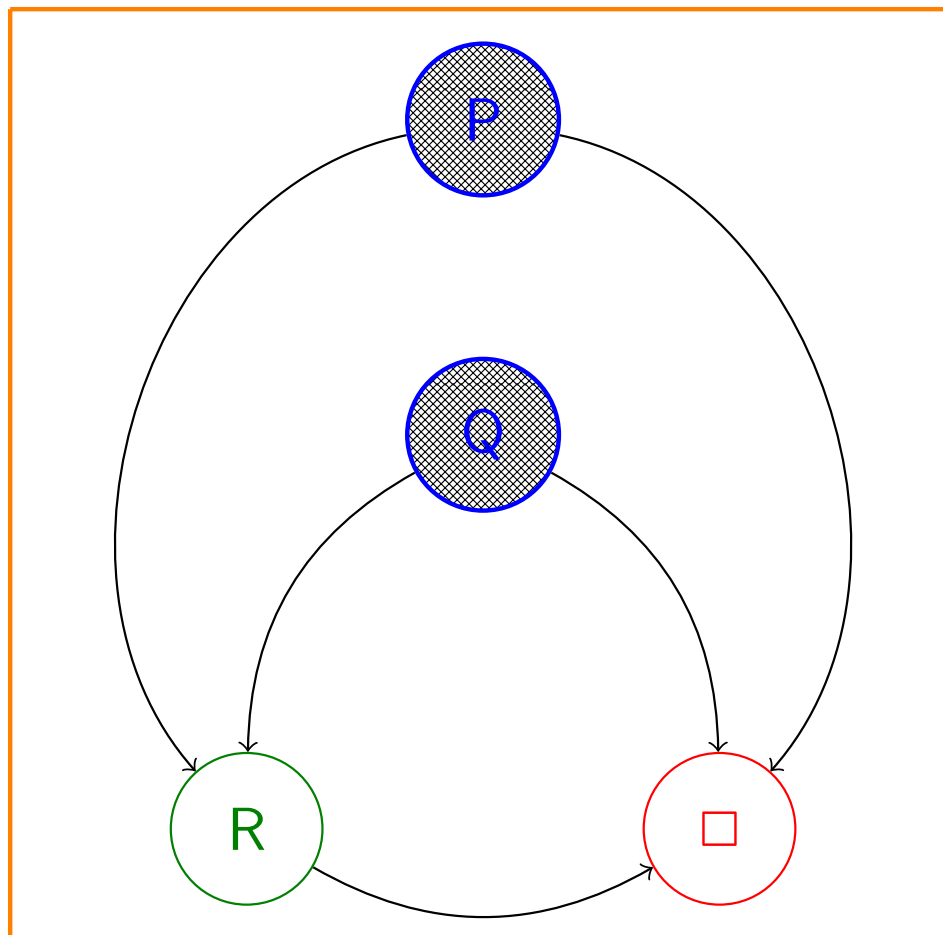
Proposition (Clause Learning). Let Δ be a set of clauses, and let G^{confl} be a conflict graph at some time point during a run of DPLL on Δ . Let $\text{choiceLits}(G^{\text{confl}})$ be the choice literals in G^{confl} . Then $\Delta \models \{\bar{l} \mid l \in \text{choiceLits}(G^{\text{confl}})\}$.

→ The negation of the choice literals in a conflict graph is a valid clause.

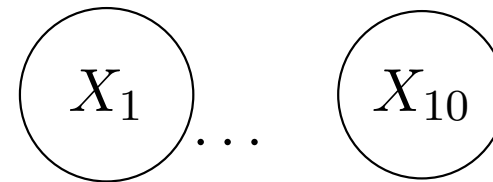
Clause Learning: Example (Redundance1)

$$\Delta = \{\{\neg P, \neg Q, R\}, \{\neg P, \neg Q, \neg R\}, \{\neg P, Q, R\}, \{\neg P, Q, \neg R\}, \\ \{X_1, \dots, X_{100}\}, \{\neg X_1, \dots, \neg X_{100}\}\}$$

Choice: $P, X_1, \dots, X_{10}, Q$. Implied: R .



$\{\neg P, \neg Q\}$



The Effect of Learned Clauses (in Redundance1)

→ What happens after we learned a new clause C ?

1. **We add C into Δ .** → Example: $C = \{\neg P, \neg Q\}$.

2. **We retract the last choice l' .**

→ Example: Retract the choice $l' = Q$.

Observation: $C = \{\bar{l} \mid l \in \text{choiceLits}(G^{\text{confl}})\}$. Before we learn the clause, G^{confl} must contain the most recent choice l' : otherwise, the conflict would have occurred earlier on. So $C = \{\bar{l}_1, \dots, \bar{l}_k, \bar{l}'\}$ where $l_1, \dots, l_k$ are earlier choices.

→ Example: $l_1 = P$, $C = \{\neg P, \neg Q\}$, $l' = Q$.

Observation: Given the earlier choices $l_1, \dots, l_k$, after we learned the new clause $C = \{\bar{l}_1, \dots, \bar{l}_k, \bar{l}'\}$, $\bar{l}'$ is now set by UP!

3. **We set the opposite choice $\bar{l}'$ as an implied literal.**

→ Example: Set $\neg Q$ as an implied literal.

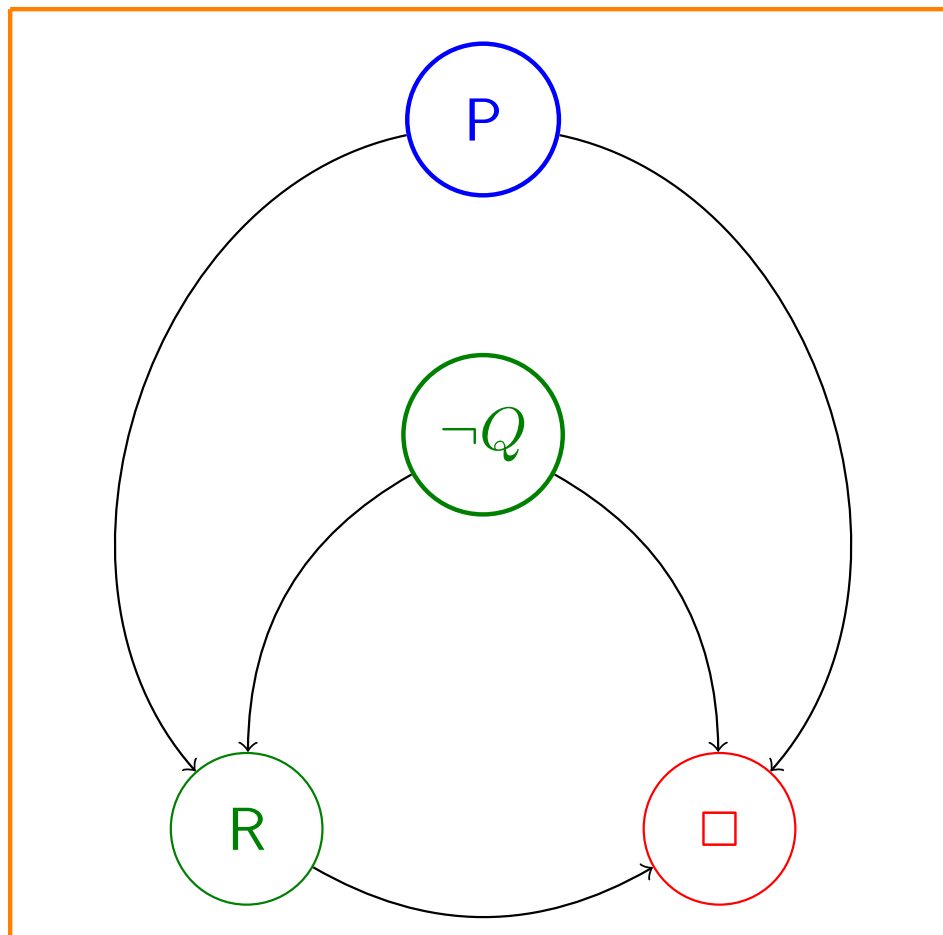
4. **We run UP and analyze conflicts.** Learned clause: earlier choices only!

→ Example: $C = \{\neg P\}$, see next slide.

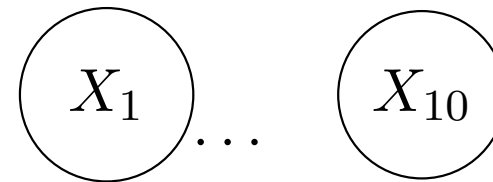
The Effect of Learned Clauses: Example (Redundance1)

$$\Delta = \{\{\neg P, \neg Q, R\}, \{\neg P, \neg Q, \neg R\}, \{\neg P, Q, R\}, \{\neg P, Q, \neg R\}, \\ \{X_1, \dots, X_{10}\}, \{\neg X_1, \dots, \neg X_{10}\}, \{\neg P, \neg Q\}\}$$

Choice: $P, X_1, \dots, X_{10}$. Implied: $\neg Q, R$.

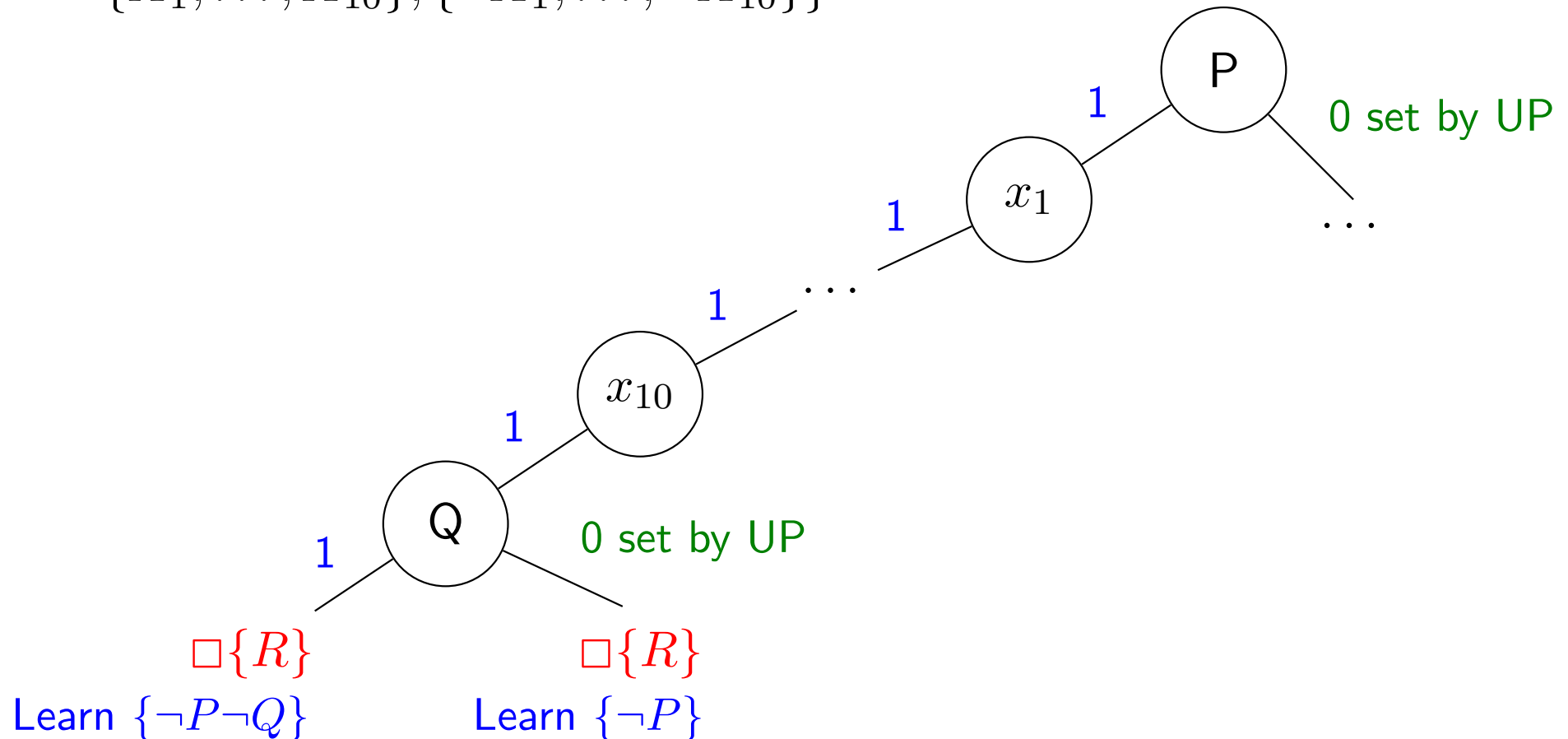


$\{\neg P\}$



NOT Same Mistakes over Again: Example (Redundance1)

$$\Delta = \{\{\neg P, \neg Q, R\}, \{\neg P, \neg Q, \neg R\}, \{\neg P, Q, R\}, \{\neg P, Q, \neg R\}, \\ \{X_1, \dots, X_{10}\}, \{\neg X_1, \dots, \neg X_{10}\}\}$$



Note: Here, the problem could be avoided by splitting over different variables. **This is not so in general!** (see next slide)

Clause Learning vs. Resolution

Remember (slide 14):

- 1 **DPLL = tree resolution**: Each derived clause C (not in Δ) is derived anew every time it is used.
- 2 There exist Δ whose **shortest tree-resolution proof is exponentially longer than their shortest (general) resolution proof**.

This is no longer the case with clause learning!

- 1 **We add each learned clause C to Δ , can use it as often as we like.**
- 2 **Clause learning renders DPLL equivalent to full resolution [??].**
(In how far exactly this is the case was an open question for ca. 10 years, so it's not as easy as I made it look here ...)

→ In particular: Selecting different variables/values to split on can *provably* not bring DPLL up to the power of DPLL+Clause Learning. (cf. slide 11, and previous slide)

“DPLL + Clause Learning”?

- General Idea: Run “DPLL + Clause Learning” by running DPLL and adding a new clause to the clause set every time we reach a conflict.
- This raises several questions:
 - How should we backtrack? Should we simply restart?
 - Should we always learn as many clauses as possible?

CDCL

```
1 #  $\Delta$  is a set of clauses
2 def CDCL( $\Delta$ )
3     while true
4          $\Delta$  = unit-propagation( $\Delta$ )
5
6         if a conflict was reached, learning  $C$ 
7             if  $C = \square$  return unsat
8             add  $C$  to  $\Delta$ 
9             backjump according to  $C$ 
10        else
11            if all variables assigned: return sat
12            pick decision literal  $l$  for some unassigned var
13             $\Delta = fix(\Delta, l)$ 
```

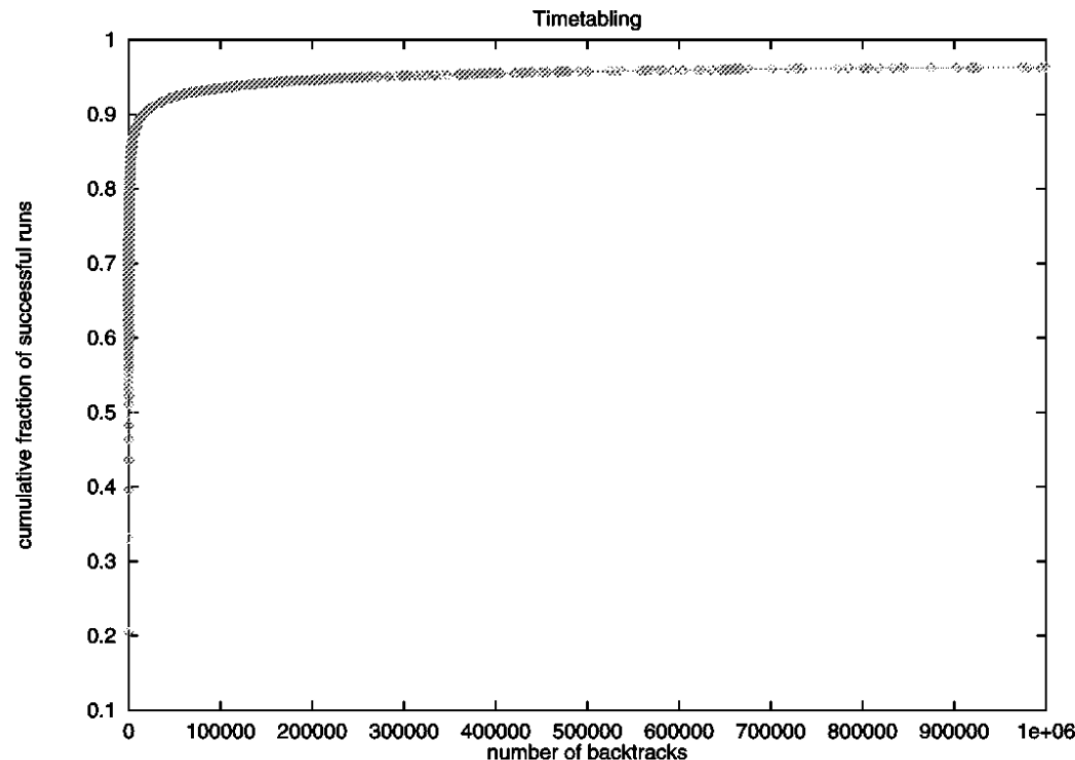
→ In this case, fix and unit-propagation do not “remove” clauses/literals from Δ . Rather, some parts of Δ are marked as true/false. When we backjump, some of the values are restored.

Other considerations

- While we only select $choiceLits(G^{confl})$, much more can be done.
- For any **cut** through G^{confl} , with $choiceLits(G^{confl})$ on the “left-hand” side of the cut and the conflict literals on the right-hand side, the literals on the left border of the cut yield a learnable clause.
- Must take care to *not learn too many clauses* ... (or forget about them)
- The choice of decision literals is extremely important (there are many heuristic criteria to optimize the algorithm)

Heavy Tailed Behaviour: Restart the Algorithm

Picture from “Heavy-Tailed Phenomena in Satisfiability and Constraint Satisfaction Problems” by Gomes et al. (2000)



→ Random restarts can help to improve the efficiency of algorithms that exhibit heavy-tailed behaviour

Damn, Where Are the Hard Problems?

Err, what?

- SAT is **NP**-hard. Worst case for DPLL is 2^n , with n propositions.
- Imagine I gave you as homework to make a formula family $\{\phi\}$ where DPLL runtime necessarily is in the order of 2^n .
 - I promise you're not gonna find this easy ... (although it is of course possible: e.g., the "Pigeon Hole Problem").
- People noticed by the early 90s that, in practice, the DPLL worst case does not tend to happen.

→ Modern SAT solvers successfully tackle practical instances where $n > 1000000$.

Damn, Where Are the Hard Problems? Ctd.

So, what's the problem? Science is about *understanding the world*.

→ Are “hard cases” just pathological outliers? Can we say something about the *typical case*?

Difficulty 1: What is the “typical case” in applications? E.g., what is the “average” Hardware Verification instance?

→ Consider precisely defined random distributions instead.

Difficulty 2: Search trees get very complex, and are difficult to analyze mathematically, even in trivial examples. Never mind examples of practical relevance ...

→ The most successful works are empirical.

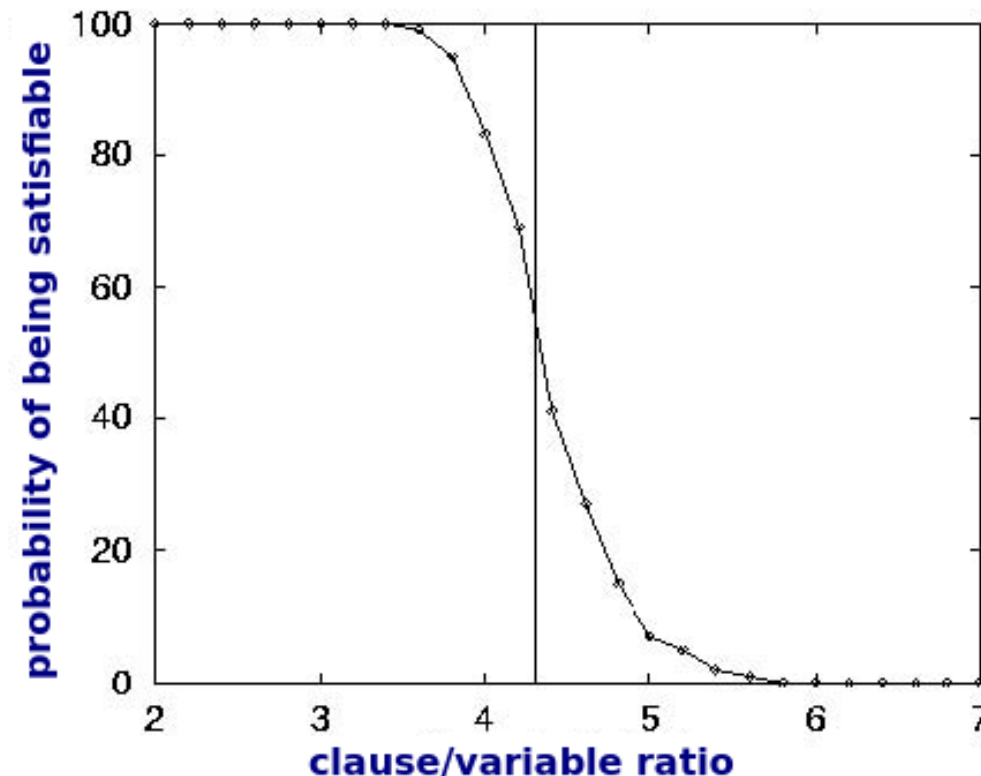
(Interesting theory is mainly concerned with *hand-crafted* formulas, like the Pigeon Hole Problem.)

Phase Transitions in SAT [?]

Fixed clause length model: Fix **clause length** k ; n **variables**. Generate m **clauses**, by uniformly choosing k variables P for each clause C , and for each variable P deciding uniformly whether to add P or $\neg P$ into C .

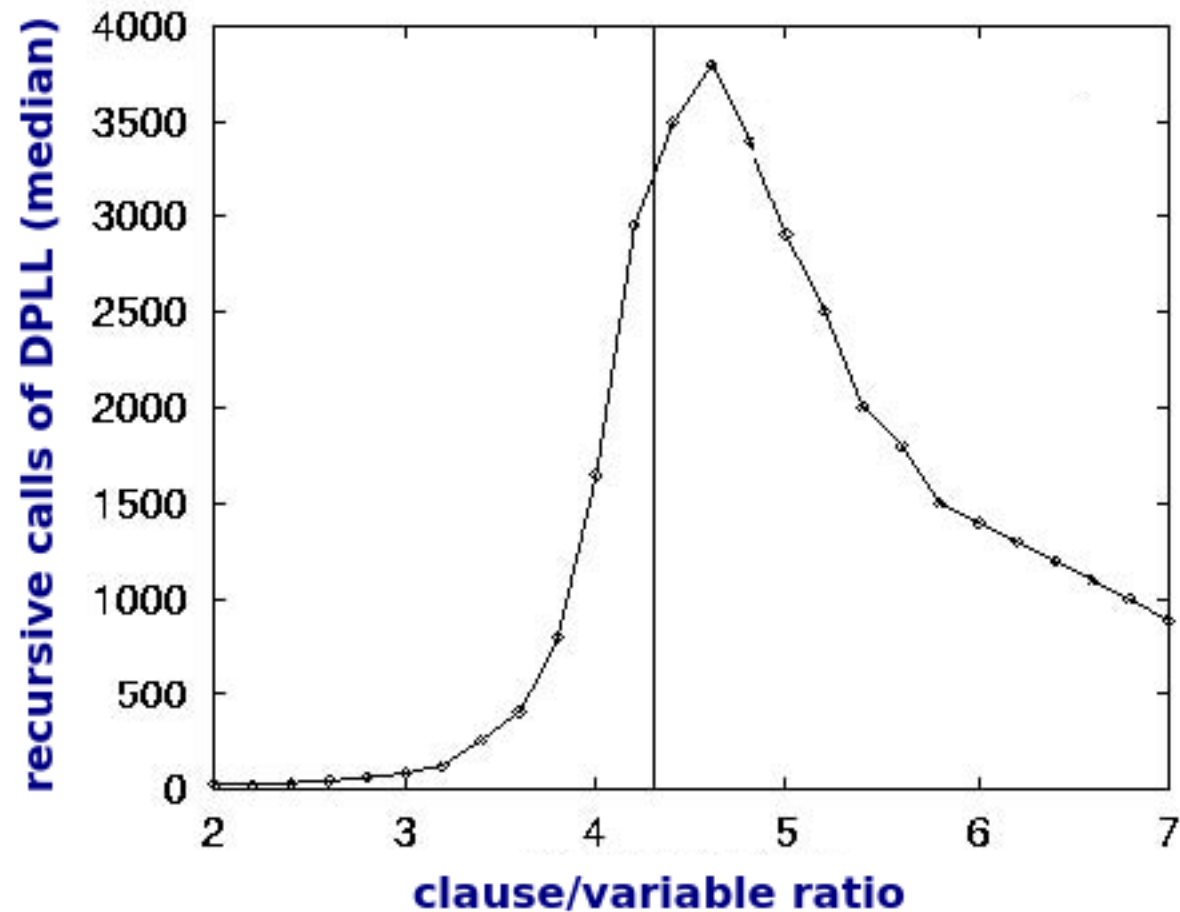
Order parameter: **Clause/variable ratio** $\frac{m}{n}$.

Phase transition: (Fixing $k = 3$, $n = 50$)



Does DPLL Care?

Oh yes, it does! Extreme runtime peak at the phase transition!



Why Does DPLL Care?

Intuitive explanation:

Under-Constrained: Satisfiability likelihood close to 1. Many solutions, first DPLL search path usually successful. (“Deep but narrow”)

Over-Constrained: Satisfiability likelihood close to 0. Most DPLL search paths short, conflict reached after few applications of splitting rule. (“Broad but shallow”)

Critically Constrained: At the phase transition, many *almost-successful* DPLL search paths. (“Close, but no cigar”)

The Phase Transition Conjecture

Conjecture: [?]

Phase Transition Conjecture

“All **NP**-complete problems have at least one **order parameter**, and the hard to solve problems are around a critical value of this order parameter. This critical value (a **phase transition**) separates one region from another, such as over-constrained and under-constrained regions of the problem space.”

→ [?] confirmed this for Graph Coloring and Hamiltonian Circuits. Later work confirmed it for SAT (see previous slides), and for numerous other **NP**-complete problems.

Why Should We Care?

Enlightenment:

- Phase transitions contribute to the fundamental understanding of the behavior of search, even if it's only in random distributions.
- There are interesting theoretical connections to phase transition phenomena in physics. (See [?] for a short summary.)

Ok, but what can we use these results for?

- **Benchmark design**: Choose instances from phase transition region.
→ Commonly used in competitions etc. (In SAT, random phase transition formulas are the most difficult for DPLL-style searches.)
- **Predicting solver performance**: Yes, but very limited because:

→ All this works only for the particular considered *distributions of instances*! Not meaningful for any other instances.

Questionnaire, ctd.

Question!

Say I encode a Wumpus problem into Δ that turns out to have clause/variable ratio 10. Which is true?

(A): Δ is very likely to be unsatisfiable.

(B): Δ is very likely to be satisfiable.

→ Neither is true. The clause/var ratio 4.3 has a meaning only for ?'s “fixed clause length model” for generating random CNF formulas.

→ For other CNF formulas, the clause/variable ratio is completely meaningless!

Extreme example: Say we generate clauses including only positive literals ...

Practical example: Many Verification problems have huge numbers of clauses but are still satisfiable.

→ For example, consider the straightforward encoding for “Exactly one of n variables $x_1, \dots, x_n$ is true.” We get the clause $\{x_1, \dots, x_n\}$ (“at least one is true”) and, for every $1 \leq i \neq j \leq n$, the clause $\{\neg x_i, \neg x_j\}$ (“at most one is true”). The clause/variable ratio is $\frac{n^2 - n + 2}{2n}$, but the formula is satisfiable.

Questionnaire, ctd.

Question!

Say I sit down tonight and write a random Δ with clause/variable ratio 1.1. Which are true?

(A): I'm bored.

(B): Δ is satisfiable.

(C): All slides for next week are prepared already.

(D): Δ is very likely to be satisfiable.

→ (A), (C): Definitely true ...

→ (B): Definitely not a certainty, for any way of generating random CNFs (unless we include only positive, or only negative, literals).

→ (D): Depends on *how* I generate the CNF. If I use ?'s methods then yes. If I use a different method, then no.

Summary

- **Implication graphs** capture how UP derives conflicts. Their analysis enables us to do **clause learning**. DPLL with clause learning is called **CDCL**. It corresponds to full resolution, not “making the same mistakes over again”.
- CDCL is state of the art in applications, routinely solving formulas with millions of propositions.
- In particular random formula distributions, typical problem hardness is characterized by **phase transitions**.

State of the Art in SAT

SAT competitions:

- Since beginning of the 90s <http://www.satcompetition.org/>
- Distinguish **random** vs. **industrial** vs. **handcrafted** benchmarks.
- Largest industrial instances: > 1000000 propositions.

State of the art is CDCL:

- **Vastly superior on handcrafted and industrial benchmarks.**
- Key techniques: **Clause Learning**! Also: Efficient implementation (UP!), good branching heuristics, random restarts, portfolios.

What about local search?

- **Better on random instances.**
- No “dramatic” progress in last decade.
- Parameters are difficult to adjust.

Topics We Didn't Cover Here

- **Variable/value selection heuristics:** A whole zoo is out there.
- **Implementation techniques:** One of the most intensely researched subjects. Famous “watched literals” technique for UP had huge practical impact.
- **Local search:** In space of all truth value assignments. GSAT (slide 9) had huge impact at the time (1992), caused huge amount of follow-up work. Less intensely researched since clause learning hit the scene in the late 90s.
- **Portfolios:** How to combine several SAT solvers effectively?
- **Random restarts:** Tackling heavy-tailed runtime distributions.
- **Tractable SAT:** Polynomial-time sub-classes (most prominent: 2-SAT, Horn formulas).
- **MaxSAT:** Assign weight to each clause, maximize weight of satisfied clauses (= optimization version of SAT).
- **Resolution special cases:** There's a universe in between unit resolution and full resolution: trade-off inference vs. search.
- **Proof complexity:** Can one resolution special case X simulate another one Y polynomially? Or is there an exponential separation (example families where X is exponentially less effective than Y)?

Reading

- The Art of Computer Programming by Donald E. Knuth, Vol 4. Section 7.2.2.2
- Handbook of Satisfiability, Hans van Maaren, Armin Biere, Toby Walsh.

References I